Abin | Track A

Enterprise policy question answering

1. Problem and domain

Employees need procedural answers from scattered company policies. An incomplete question can lead to the wrong rule: travel and medical claims, for example, have different deadlines. This Scenario S2 prototype answers from retrieved policy evidence, asks the requester to clarify missing circumstances, and declines to invent absent information.

Four synthetic policies contain 11 sections covering expenses, leave, system access, and security incidents. A deterministic generator emits fixed templates and values without random sampling or model training. Four development cases and eight held-out questions exercise direct answers, combined facts, ambiguity, and missing information. The clean synthetic corpus is suitable for demonstrating the workflow, but cannot establish real-world reliability.

2. Approach and algorithm decisions

Three roles separate evidence gathering, drafting, and review. The researcher selects a query, calls the search tool, assesses relevance, and chooses writing or clarification. The writer drafts from the evidence. The verifier checks support and completeness, then accepts, requests retrieval or rewriting, asks for clarification, or requires abstention. Distinct prompts and Pydantic outputs define their responsibilities; JSONL events record their handoffs. A manager would add calls without a necessary separate role.

A single-LLM retrieval pipeline can answer simple lookups; this agent design makes scope review and targeted correction explicit. Classical classification was rejected because fixed labels cannot synthesize open-ended procedural answers. Separate roles add latency and share model weaknesses, so they do not guarantee higher accuracy.

LangGraph provides state, routing through Command, and interrupt/resume with checkpoints. CrewAI is viable, but explicit transitions fit this bounded workflow. A custom loop would need its own checkpoint machinery. When a claim type is missing, the graph pauses for the requester to confirm the scope before continuing. This human checkpoint controls interpretation rather than asking the employee to certify policy facts.

Section-level chunks preserve headings, rules, and source IDs; each must fit MiniLM's 256-token input limit. CPU MiniLM embeddings and Chroma cosine top-3 search keep retrieval local. Fixed-size splitting could separate a rule from its context. Keyword-only search can miss paraphrases; hybrid search and reranking add complexity before retrieval failures justify them. FAISS would need separate metadata and persistence handling.

Ollama runs one Qwen 3.5 4B model sequentially for all roles; ChatOllama supplies structured outputs and tool binding. A larger model leaves less headroom on the development laptop's 8 GB GPU, while hosted inference adds an external dependency. Streamlit calls the graph directly; a separate API is unnecessary for this local demo. Citations show only file name and section on hover, while full traces stay in files.

Current limits allow three searches, three correction cycles, and three clarification exchanges per run, plus three output-repair retries per model output. A separate 40-step limit applies per invocation. Citation validation and scope checks reduce silent failures; bounded loops prevent repeated correction from continuing indefinitely.

3. Results and error analysis

The frozen evaluation covered eight questions: three direct, two requiring multiple sections, one ambiguous request, and two unsupported requests. Expected facts and gold source IDs were used only for scoring. Automated clarification replies were explicitly labelled simulated and supplied only the missing circumstance.

Metric	Recorded result
Correct responses	7/8 (87.5%)
Supported policy claims in attempted answers	16/16
Correct abstentions on unsupported requests	2/2
Unnecessary abstentions on answerable requests	0/6
Mean elapsed time per question	8.21 seconds
Total model calls across eight questions	34

These measurements describe the earlier evaluated implementation, prompt version 1, and one-correction/one-clarification/one-repair limits. They are not a fresh evaluation of the current prompt version 2, ChatOllama client, routing, budgets, or interface. The original scores remain unchanged after subsequent fixes.

Correctness requires every requested fact or justified abstention, with no unsupported extra assertion. Claim support checks the cited policy text and excludes abstention/process commentary. Labels were reviewed by the coding assistant, not an independent human or blinded evaluator. Timing covers one warmed local run and excludes human waiting and initial downloads. Some facts overlap between development and evaluation, so this is a question-level holdout rather than unseen-document testing.

The failed response declined to invent an absent international meal allowance, but appended an unsupported assertion that a relevant search had not occurred. The trace showed that it had. The cause was finalization copying diagnostic review feedback into the answer after the draft had been checked. Finalization now uses a fixed, plain-language fallback and records internal reasons in the trace. Regression tests cover this path without changing the historical 7/8 score.

Other development failures included listing alternatives instead of clarifying, inventing section IDs, and requesting unnecessary revisions. Scope checks, retrieved-ID constraints, and narrower review instructions address these behaviors. The current automated suite has 32 passing tests covering graph routes, clarification/resume, cancellation, correction limits, citation checks, model-output repair, and UI behavior; Ruff lint and formatting checks pass. These checks use fake agents and mocked model transport and do not replace live-model evaluation. Saved complete traces document a success and an ambiguous claim resolved by a real user's reply.

4. Production considerations and limitations

Enterprise deployment must enforce document permissions during retrieval and restrict access to full traces, which can contain questions and policy text. Durable checkpoint storage is needed because the current in-memory state is lost on restart. Evaluated prompt/index versions and failure/latency monitoring would support controlled updates.

The prototype assumes short English text policies and one local user, with clarification subject to the shared search budget. Four synthetic policies and eight questions do not cover scans, conflicting versions, complex exceptions, prompt injection, or corpus scale. The same model writes and reviews, so agreement is not independent evidence of correctness. With more time, evaluate real permissioned policies and independently reviewed labels, then test a stronger model or hybrid retrieval when observed failures justify the added cost and complexity.